Explicacion del codigo MATLAB[0.5cm] Metodos Numericos

Índice

1. **Metodo de Biseccion** **2**
2. **Metodo de Newton-Raphson** **3**
3. **Metodo de la Secante** **4**
4. **Metodo de Punto Fijo** **6**
5. **Metodo de Regula Falsi** **7**
6. **Estructura general del programa** **9**

1. Metodo de Biseccion

El metodo de biseccion es un metodo numerico cerrado que permite aproximar una raiz de una funcion a partir de un intervalo inicial $[a, b]$. Para aplicar este metodo se necesita que la funcion sea continua en el intervalo y que los valores $f(a)$ y $f(b)$ tengan signos diferentes. De esta manera se garantiza la existencia de al menos una raiz dentro del intervalo.

En el codigo MATLAB, primero se define la funcion mediante una funcion anonima. Por ejemplo, para el primer ejercicio se utiliza:

```
f=@(x)x.^3-7*x+4;
```

Luego se establecen los extremos del intervalo y el numero de iteraciones:

```
a=-3;
b=-2;
n=4;
```

El programa calcula inicialmente los valores $f(a)$ y $f(b)$ para observar el comportamiento de la funcion en los extremos del intervalo. Posteriormente, se utiliza un ciclo `for` para realizar las iteraciones del metodo.

En cada iteracion se calcula el punto medio mediante:

$$x_m = \frac{a+b}{2}$$

En MATLAB esta operacion se realiza mediante:

```
xm=(a+b)/2;
```

Despues se evalua la funcion en el punto medio:

```
fxm=f(xm);
```

El programa calcula el producto $f(a)f(x_m)$ para determinar en que mitad del intervalo se encuentra la raiz. Si:

$$f(a)f(x_m) < 0$$

la raiz se encuentra entre a y x_m, por lo que se actualiza el extremo derecho:

```
b=xm;
```

Si:

$$f(a)f(x_m) > 0$$

la raiz se encuentra entre x_m y b, por lo que se actualiza el extremo izquierdo:

```
a=xm;
```

Si el producto es igual a cero, significa que x_m es una raiz exacta y el proceso puede detenerse.

El codigo tambien calcula el error aproximado porcentual a partir de dos puntos medios consecutivos:

$$E_a = \left| \frac{x_{m,nuevo} - x_{m,anterior}}{x_{m,nuevo}} \right| 100$$

Para la primera iteracion no se calcula este error debido a que todavia no existe un punto medio anterior. Por ello se utiliza:

```
xm_anterior=NaN;
```

El segundo ejercicio del metodo de biseccion utiliza la funcion:

$$f(x) = x - e^{1/x}$$

con el intervalo $[1, 2]$ y una tolerancia de 10^{-4}. En este caso, el codigo calcula inicialmente el numero teorico de iteraciones mediante:

$$n = \left\lceil \log_2 \left(\frac{b-a}{T} \right) \right\rceil$$

El valor obtenido sirve como referencia teorica. Sin embargo, posteriormente se ejecutan exactamente tres iteraciones porque el ejercicio solicita realizar tres iteraciones.

En ambos ejercicios, la funcion `fprintf` permite mostrar en pantalla los valores utilizados durante cada iteracion, incluyendo los extremos del intervalo, el punto medio, la evaluacion de la funcion, el producto de signos, el error y el nuevo intervalo.

2. Metodo de Newton-Raphson

El metodo de Newton-Raphson es un metodo numerico abierto utilizado para aproximar las raices de una funcion. A diferencia de la biseccion, no necesita mantener un intervalo que contenga la raiz. El metodo utiliza un valor inicial y la derivada de la funcion.

La formula utilizada es:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

En el primer ejercicio se trabaja con:

$$f(x) = \cos(x) - 3x$$

y su derivada:

$$f'(x) = -\sin(x) - 3$$

Estas funciones se definen en MATLAB mediante:

```
f=@(x)cos(x)-3*x;
df=@(x)-sin(x)-3;
```

El valor inicial utilizado es:

$$x_0 = \frac{\pi}{8}$$

que en MATLAB se representa mediante:

```
x=pi/8;
```

Tambien se establece una tolerancia:

$$T = 10^{-5}$$

En cada iteracion, el programa calcula el valor de la funcion y de su derivada:

```
fx=f(x);
dfx=df(x);
```

Posteriormente aplica directamente la formula de Newton-Raphson:

```
x_nuevo=x-fx/dfx;
```

Despues se calcula el error absoluto entre el nuevo valor y el valor anterior:

$$E = |x_{nuevo} - x|$$

El programa compara este error con la tolerancia establecida. Si:

$$E < T$$

se considera que se ha alcanzado una aproximacion suficientemente precisa y el proceso termina mediante la instruccion `break`. Si el error todavia es mayor o igual que la tolerancia, el nuevo valor pasa a ser el punto inicial de la siguiente iteracion.

El segundo ejercicio utiliza la funcion:

$$f(x) = x^3 + 2x^2 + 10x - 20$$

y su derivada:

$$f'(x) = 3x^2 + 4x + 10$$

El valor inicial es $x_0 = 1$ y la tolerancia es 10^{-3}. El procedimiento utilizado es el mismo: evaluar la funcion y su derivada, aplicar la formula de Newton-Raphson, calcular el error y comprobar si se cumple la tolerancia.

La variable `maxit=100` establece un limite de 100 iteraciones para evitar que el ciclo continue indefinidamente en caso de que no se alcance la tolerancia.

3. Metodo de la Secante

El metodo de la secante es un metodo numerico abierto que permite aproximar una raiz utilizando dos valores iniciales. A diferencia del metodo de Newton-Raphson, no requiere calcular explicitamente la derivada de la funcion.

La formula utilizada es:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

En el primer ejercicio se utiliza:

$$f(x) = \cos(x) - x$$

con los valores iniciales:

$$x_0 = 0.5$$

y

$$x_1 = \frac{\pi}{4}$$

En MATLAB estos valores se establecen mediante:

```
x0=0.5;
x1=pi/4;
```

En cada iteracion, el programa calcula:

$$f(x_0)$$

y

$$f(x_1)$$

mediante:

```
fx0=f(x0);
fx1=f(x1);
```

Luego aplica la formula de la secante:

```
x2=x1-fx1*(x1-x0)/(fx1-fx0);
```

El nuevo valor obtenido se denomina x_2. Posteriormente se calcula la funcion en este nuevo punto y el error aproximado porcentual:

$$E_a = \left| \frac{x_2 - x_1}{x_2} \right| 100$$

Una vez finalizada la iteracion, se actualizan los valores:

```
x0=x1;
x1=x2;
```

De esta forma, los dos ultimos valores calculados se utilizan en la siguiente iteracion.

El segundo ejercicio utiliza la funcion:

$$f(x) = x^3 + 2x^2 + 10x - 20$$

con:

$$x_0 = 0$$

y

$$x_1 = 1$$

En este caso se establece una tolerancia de 10^{-3}. El programa calcula el nuevo valor mediante el metodo de la secante y posteriormente obtiene el error absoluto:

$$E = |x_2 - x_1|$$

Si el error es menor que la tolerancia, el ciclo se detiene. En caso contrario, se actualizan x_0 y x_1 para continuar con una nueva iteracion.

4. Metodo de Punto Fijo

El metodo de punto fijo transforma una ecuacion de la forma:

$$f(x) = 0$$

en una ecuacion equivalente:

$$x = g(x)$$

A partir de un valor inicial x_0, se genera una sucesion mediante:

$$x_{n+1} = g(x_n)$$

En el primer ejercicio se utiliza:

$$g(x) = \sqrt{3x + 1}$$

y el valor inicial:

$$x_0 = 3$$

La funcion se define en MATLAB mediante:

```
g=@(x)sqrt(3*x+1);
```

El programa realiza cuatro iteraciones. En cada una calcula el nuevo valor:

$$x_{nuevo} = g(x)$$

que en MATLAB corresponde a:

```
x_nuevo=g(x);
```

Despues se calcula el error absoluto:

$$E = |x_{nuevo} - x|$$

y el error aproximado porcentual:

$$E_a = \left| \frac{x_{nuevo} - x}{x_{nuevo}} \right| 100$$

Finalmente, el valor nuevo se convierte en el valor utilizado para la siguiente iteracion:

```
x=x_nuevo;
```

El segundo ejercicio utiliza la funcion de iteracion:

$$g(x) = \frac{9x^{10} + 1}{10x^9 - 2}$$

con el valor inicial:

$$x_0 = 1.2$$

y una tolerancia de:

$$T = 10^{-4}$$

En cada iteracion se calcula el nuevo valor mediante:

```
x_nuevo=g(x);
```

Luego se obtiene el error absoluto y se compara con la tolerancia. Si el error cumple:

$$E < T$$

el algoritmo finaliza. De lo contrario, continua con la siguiente iteracion.

El ciclo esta limitado por `maxit=100`, lo que permite establecer un numero maximo de iteraciones.

5. Metodo de Regula Falsi

El metodo de Regula Falsi, tambien denominado metodo de la falsa posicion, es un metodo numerico cerrado que conserva un intervalo $[a, b]$ donde existe un cambio de signo. A diferencia de la biseccion, el nuevo punto no se obtiene mediante el punto medio, sino mediante la interseccion de la recta secante con el eje x.

La formula utilizada es:

$$x_m = b - f(b)\frac{b-a}{f(b)-f(a)}$$

En el primer ejercicio se utiliza:

$$f(x) = e^x + 4x^3 - 5$$

con el intervalo inicial:

$$[0, 1]$$

La funcion se define mediante:

```
f=@(x)exp(x)+4*x.^3-5;
```

En cada iteracion se calculan los valores:

$$f(a)$$

y

$$f(b)$$

mediante:

```
fa=f(a);
fb=f(b);
```

Posteriormente se calcula el nuevo punto mediante:

```
xm=b-fb*(b-a)/(fb-fa);
```

Una vez obtenido x_m, se calcula:

$$f(x_m)$$

y se analiza el producto:

$$f(a)f(x_m)$$

Si:

$$f(a)f(x_m) < 0$$

la raiz se encuentra entre a y x_m, por lo que se actualiza:

```
b=xm;
```

Si:

$$f(a)f(x_m) > 0$$

la raiz se encuentra entre x_m y b, por lo que se actualiza:

```
a=xm;
```

Si:

$$f(x_m) = 0$$

se ha encontrado una raiz exacta y el proceso puede detenerse.

El codigo tambien calcula el error aproximado porcentual utilizando dos aproximaciones consecutivas:

$$E_a = \left| \frac{x_m - x_{m,anterior}}{x_m} \right| 100$$

Para la primera iteracion no se calcula este error porque no existe una aproximacion anterior.

El segundo ejercicio utiliza:

$$f(x) = x^{10} - 1$$

con el intervalo inicial:

$$[0, 1.3]$$

El procedimiento es el mismo. Se calcula la falsa posicion, se evalua la funcion en el nuevo punto, se determina el nuevo intervalo mediante el cambio de signo y se repite el proceso durante tres iteraciones.

6. Estructura general del programa

Antes de ejecutar los metodos, el programa utiliza:

```
clc;
clear;
format long;
```

La instruccion `clc` limpia la ventana de comandos de MATLAB. La instruccion `clear` elimina las variables almacenadas en el espacio de trabajo. Finalmente, `format long` permite mostrar los resultados numericos con una mayor cantidad de cifras decimales.

La funcion `fprintf` se utiliza constantemente para mostrar informacion detallada durante la ejecucion. Esto permite visualizar los valores iniciales, las formulas utilizadas, las sustituciones, los resultados intermedios, los errores y los resultados finales.

El programa esta organizado mediante secciones utilizando la instruccion:

```
%%
```

Estas secciones permiten separar claramente los cinco metodos numericos y facilitan la lectura y ejecucion del codigo dentro de MATLAB.

En conjunto, el programa implementa cinco metodos numericos para la aproximacion de raices: Biseccion, Newton-Raphson, Secante, Punto Fijo y Regula Falsi. Cada metodo muestra paso a paso el procedimiento matematico utilizado y proporciona los valores necesarios para analizar las aproximaciones obtenidas.